

Dynamic Memory Allocation

81. Student Record Management using malloc()

Write a C program to:

- A. Dynamically allocate memory for 10 students.
- B. Each student has: Roll No, Name, CGPA.
- C. Accept data for all students.
- D. Display:
 - a. Student with highest CGPA
 - b. Average CGPA of class
- E. Properly free the allocated memory.

82. Dynamic Array Expansion using realloc()

Initially allocate memory for 5 integers.

Assume user enters: 10, 20, 30, 40, 50

Then:

- A. Use `realloc()` to expand array size to 10 integers.
- B. Add values: 60, 70, 80, 90, 100
- C. Display:
 - a. Complete array
 - b. Sum of all elements
 - c. Largest element

Handle the case when `realloc()` returns NULL.

Dynamic Memory Allocation

83. Dynamic Matrix Creation

Write a program to:

A. Dynamically create a 3×3 matrix using `malloc()`.

B. Assume matrix values:

```
1 2 3
4 5 6
7 8 9
```

C. Calculate:

a. Sum of each row

b. Sum of each column

D. Free the entire matrix properly.

84. `calloc()` with Structure Array

Define:

```
struct Employee {
    int id;
    float salary;
};
```

A. Use `calloc()` to allocate memory for 6 employees.

B. Assume salary values.

C. Increase salary by 10% for employees earning less than 50,000.

D. Display updated records.

Dynamic Memory Allocation

85. Dynamic String Handling

Write a program to:

- A. Allocate memory dynamically for a string of size 20.
- B. Assume input: "National Institute"
- C. Use `realloc()` to increase size to 40.
- D. Append " of Technology".
- E. Display final string.
- F. Free memory safely.

86. Dangling Pointer Detection

Write a program that:

- A. Allocates memory for an integer.
- B. Frees the memory.
- C. Tries to access the freed memory (observe behaviour).
- D. Then correct the program by setting pointer to NULL.

Explain why dangling pointer is dangerous.

Dynamic Memory Allocation

87. Memory Leak Demonstration and Correction

Given:

```
int *ptr = (int *)malloc(sizeof(int));  
ptr = NULL;
```

- A. Explain why this causes memory leak.
- B. Modify the program to fix the issue.
- C. Use tools (like valgrind – optional) to verify memory leak.

88. Create a structure having the following members:

- A. `int roll`
- B. `float marks`
- C. Pointer to same structure type named `partner`

Task:

- A. Define the structure using the above members.
- B. Dynamically allocate memory for **2 student records** using `malloc()`.
- C. Assume the following data:
 - a. Roll: 101, Marks: 78.5
 - b. Roll: 102, Marks: 88.0
- D. Link them such that:
 - a. First student's `partner` points to second student.

Dynamic Memory Allocation

- b. Second student's partner points to first student.

E. Display:

- a. Each student's roll number and marks.
- b. Their partner's roll number.

F. Calculate:

- a. Average marks of both students.
- b. Print the roll number of the student who scored higher.

89. Dynamic Memory Failure Handling

Write a program that:

- A. Tries to allocate a very large memory block using `malloc()`.
- B. If allocation fails, print "Memory Allocation Failed" and exit.
- C. Otherwise, initialize memory and free it.

90. Combined `malloc()`, `calloc()`, `realloc()` Program

Create a menu-driven program:

1. Allocate memory using `malloc()`
2. Allocate memory using `calloc()`
3. Resize memory using `realloc()`

Dynamic Memory Allocation

4. Free memory

5. Exit

The program should:

A. Prevent double free

B. Prevent dangling pointer

C. Handle NULL pointer cases properly